

Technical Blog

In October 2021, Apache HTTP Server pushed out an emergency fix for a vulnerability that turned out to be embarrassing not because it was complicated, but because it wasn't. CVE-2021-41773, and the incomplete patch that followed it, CVE-2021-42013, let an attacker read files off the server's filesystem with nothing more than a specially formatted URL — no login, no user interaction, just one HTTP request. On servers where `mod_cgi` was also enabled, the same flaw could be pushed into full remote code execution. Both were rated 9.8 out of 10 on CVSS, and within hours of Apache's own advisory going public, mass scanning for vulnerable servers was already underway.

What the vulnerability is

At its core, this is a path traversal bug. Apache is supposed to keep requests confined to the document root you've configured — you ask for `/images/logo.png`, and it serves exactly that file and nothing outside that directory. The vulnerability broke that boundary. By encoding a single dot in a URL as `%2e` instead of a literal period, an attacker could smuggle a `"../"` sequence past Apache's directory checks and walk the request path out to the filesystem root, then back down into files like `/etc/passwd`.

Where it got worse was CGI. If a server had `mod_cgi` loaded — common on older shared-hosting setups — the same escape technique could route a request into `/bin/sh` instead of a file, and whatever was sent in the POST body got executed as a shell command. That's the difference between reading a password file and running arbitrary commands on the box.

Why it occurs

The bug lives in `ap_normalize_path()`, inside `server/util.c`, whose job is to collapse a raw request path down to something safe — turning `/a/b/../c` into `/a/c`, and rejecting anything that climbs above the allowed directory. In 2.4.49, the order of operations got mixed up: Apache checked for traversal sequences first, and only afterward decoded percent-encoded characters. So a segment written as `..%2e` passed the traversal check looking harmless, got decoded into `..` after the fact, and was never re-checked.

Apache's fix in 2.4.50 explicitly rejected any path containing `%2e`, but the check only decoded once — attackers found that double-encoding the percent sign itself slipped through the single-pass filter, then unfolded into `..` once Apache finished processing the request. That's CVE-2021-42013, covered below. The real fix, in 2.4.51, decoded the path fully before doing any traversal checks, rather than pattern-matching specific encodings.

This bug alone doesn't hand you the keys, though. Apache's default config denies access to the filesystem root, so a fresh install isn't exploitable even with the flawed function underneath. It needs a second ingredient — an admin who has set Require all granted on the root directory, usually to fix some unrelated permissions headache. That combination wasn't rare in 2021.

How it can be reproduced

I built this as an isolated Docker lab rather than touching anything internet-facing — two containers, one running the vulnerable httpd:2.4.49 image from Docker Hub, configured with mod_cgi on and the permissive directory setting above, and a second running httpd:2.4.51 with hardened defaults for comparison. Both bind only to 127.0.0.1.

Against the vulnerable container, GET /icons/.%2e/%2e/%2e/%2e/etc/passwd returns a 200 with the contents of /etc/passwd. Chaining into the CGI handler, a POST to /cgi-bin/.%2e/.%2e/bin/sh with a body of echo; id returns the output of the id command, confirming code execution rather than just a file read. The same requests against the patched container return a flat 400 Bad Request both times.

Technical details of the attack

The two request patterns above are the attack in miniature. The first, .%2e, is the original 2.4.49 flaw — a single percent-encoded dot defeats the traversal check, because that check runs before decoding happens. Stack enough of those segments together and the path walks up out of the /icons/ alias, through the filesystem root, and back into whatever file you name.

The second pattern, .%%32%65, is the 42013 bypass against 2.4.50's patch. Here %25 (a literal percent sign) is itself encoded, so the filter scans for %2e and doesn't find it — it isn't there yet. Only after that filter passes does a second decoding step turn %32 and %65 back into 2 and e, reassembling %2e and then . just like before — same underlying flaw, one more layer of encoding than the patch checked for.

On the CGI side, the mechanism differs from a simple file read. The traversal path resolves to /bin/sh instead of a static file, and because mod_cgi is loaded, Apache executes it as a script, passing the HTTP request body straight to the shell's standard input — whatever command is in that body runs with the privileges of the Apache worker process.

Detection

Detection works on two levels. The lazy version is a banner check — Apache reports its own version in the Server response header unless told not to, so anything reporting 2.4.49 or 2.4.50 is worth flagging. That's not reliable alone, though, since admins who turn on ServerTokens Prod to

hide the version string would make a vulnerable box look clean on a banner check.

The more trustworthy method is an active, non-destructive probe: send the traversal pattern at a harmless file — `/etc/issue` instead of `/etc/passwd` — and watch whether you get a 200 (vulnerable) or a 400 (patched). That's the approach used in this lab's detection script, and it doesn't depend on the server being honest about its own version.

Remediation

There's really one correct fix: upgrade to Apache 2.4.51 or later, the only version where the decode-then-validate ordering was actually corrected rather than patched around. If you can't upgrade immediately, setting `Require all denied` explicitly on `<Directory />` removes the misconfiguration that makes the traversal exploitable, and disabling `mod_cgi` where it isn't needed closes off the RCE path. Turning on `ServerTokens Prod` won't stop a real attacker, but it does deny easy reconnaissance to anyone scanning banners for easy targets.

Key takeaways

What stands out most about this CVE isn't the traversal bug itself — path traversal bugs are old news. It's how a single reordering of two operations inside one function created a critical, unauthenticated RCE in one of the most widely deployed server platforms on the planet, and how the first attempt at a fix still wasn't enough. Patching by blocking a known payload pattern, rather than fixing the logical flaw underneath it, tends to just move the goalposts for whoever's looking for the next bypass. The people who found CVE-2021-42013 didn't need a new bug — they just needed to double-encode the old one.

Sources

The technical details above draw on Apache's own security advisory for the 2.4.x branch, the CVE records for CVE-2021-41773 and CVE-2021-42013 on NVD, CISA's advisory AA21-281A, and public write-ups from Qualys and Huntress Labs. All reproduction, evidence, and analysis here were carried out independently in an isolated lab, in my own words.